

执行摘要

本次实验共评估 5 个场景，模型为「场景评测 Mock 388b64」，每个场景均包含 10 条测试数据，合计 50 条。所有实验状态均为 completed，数据覆盖率为 100%，无 provider_errors 和 metric_errors。

- 整体平均准确率: $(0.6 + 0.0 + 0.5 + 0.5 + 0.5) / 5 = 0.42$
- 最佳场景: QA, 准确率 0.6
- 最差场景: CLASS, 准确率 0.0
- CODING、AGENT、GEN 准确率均为 0.5
- 所有失败样本的共性是: 模型仅输出前缀、重复输入或残缺代码，未生成有效目标内容，表明问题具有系统性。

性能分析

各场景关键指标如下:

场景	评估指标	准确率	行数	已评分	覆盖率	失败率	平均延迟
QA	exact_match_ci	0.6	10	10	1.0	0.0	189.6 ms
CLASS	exact_match_ci	0.0	10	10	1.0	0.0	274.5 ms
CODING	code_pass	0.5	10	10	1.0	0.0	175.9 ms
AGENT	tool_call	0.5	10	10	1.0	0.0	203.8 ms
GEN	f1_token	0.5	10	10	1.0	0.0	213.2 ms

- QA 表现最好，10 条中通过 6 条。
- CLASS 表现最差，准确率为 0.0，即所有已评分样本均未通过。
- CODING、AGENT、GEN 均仅有 50% 的样本通过。
- 平均延迟约为 211.4 ms；CLASS 延迟最高（274.5 ms），但准确率最低，说明延迟与效果无正相关。

成本分析

所有实验的 total_cost 均为 0.0，未产生直接费用。Token 消耗如下:

场景	Token 数
QA	43
CLASS	92
CODING	247
AGENT	228
GEN	109
合计	719

- 平均每个场景消耗 143.8 tokens。
- CODING 消耗 token 最多（247），主要原因是输出代码片段较长。

- CLASS 消耗 92 tokens，但准确率为 0.0，说明当前 token 消耗没有转化为有效答案。
- 由于无 provider 错误和 metric 错误，也未触发额外重试成本。

失败分析

5 个场景均存在得分 0.0 的失败样例，失败模式高度一致：

QA

- 失败行为：输出固定前缀 答案：
- 示例：
 - 输入「退款多久到账？」期望 1-3个工作日，实际输出 答案：
 - 输入「支持货到付款吗？」期望 目前暂不支持，实际输出 答案：
- 共 4 条失败，均未生成实际答案内容。

CLASS

- 准确率为 0.0，即全部样本未通过。
- 失败行为：输出固定前缀 类别：
- 示例：
 - 输入 NBA 新闻，期望 体育，实际输出 类别：
 - 输入央行降准新闻，期望 财经，实际输出 类别：
- 模型未输出任何类别标签。

CODING

- 失败行为：仅输出函数中的 return 行，而不是完整函数定义。
- 示例：
 - sum_list 期望完整函数，实际输出 return total
 - bubble_sort 期望完整函数，实际输出 return arr
- 这些输出无法通过单元测试，因此 code_pass 为 0。

AGENT

- 失败行为：直接重复用户输入，没有调用任何工具。
- 示例：
 - 输入「帮我查一下账户 A1001 的余额」，期望调用 check_balance，实际输出为原始问题文本。
 - 输入「请把 500 元从 A1001 转到 A1002」，期望调用 transfer_money，实际输出仍为原始问题文本。
- 模型未生成任何结构化工具调用参数。

GEN

- 失败行为：输出固定前缀 摘要：
- 示例：
 - 输入公司营收公告新闻，期望生成摘要，实际仅输出 摘要：

- 5 条失败样本均只输出前缀，没有摘要内容。

综上，失败并非随机错误，而是模型在生成答案时“只输出格式前缀/输入复述/代码片段，未继续生成核心内容”的系统性问题。failure_rate 为 0 表示没有发生服务或评分异常，但语义和格式层面的失败大量存在。

建议

1. 修正 Prompt 指令

- 明确要求「不要输出前缀，直接输出最终答案」。
- 在 QA、GEN 中禁止输出 答案：、摘要： 等占位前缀。
- 在 CLASS 中要求直接输出分类标签，例如 体育 而不是 类别：体育。

2. 针对 AGENT 使用结构化工具调用

- 当前 AGENT 输出为原始问题文本，无法完成工具调用。
- 应接入 function calling / tool call 格式，要求输出 function_name 和 arguments，并进行 JSON 校验。

3. 针对 CODING 增加完整代码约束

- 要求输出完整函数体，不能只输出 return 语句。
- 可要求用 Markdown 代码块包裹，并在评估前提取完整函数。

4. 增加输出完整性检测

- 对仅输出前缀、仅复述输入的样本，直接判定为失败并触发一次重试。
- 重试 prompt 可追加「你的回答不完整，请补全」等提示。

5. 扩展样本量

- 每个场景仅 10 条数据，统计波动较大；建议扩充到至少 50 条/场景，以便获得更稳定的准确率估计。

下一步行动

1. 导出全部 50 条模型的完整输出，逐一确认失败模式是否全部为“截断/前缀/复述”问题。
2. 优先修复 CLASS：该场景准确率为 0，影响最大；先调整 prompt，再验证输出标签是否恢复。
3. 为 QA、GEN 增加前缀过滤和后处理，把 答案：、摘要： 剥离后参与评分。
4. 为 AGENT 配置工具函数定义和结构化输出验证，重跑 tool_call 场景。
5. 为 CODING 场景引入代码执行容器，确保输出函数可执行并通过单元测试。
6. 修复后重新运行全部实验，对比各场景准确率、延迟和 token 消耗，确认改进效果。